

USING GRAPH NEURAL NETWORK TO SOLVE THE TRAVELING SALESMAN PROBLEM

Pingbang Hu, Jonathan Moore, Yi Zhou, Shubham Kumar Pandey, Anuraag Ramesh

University of Michigan



Abstract

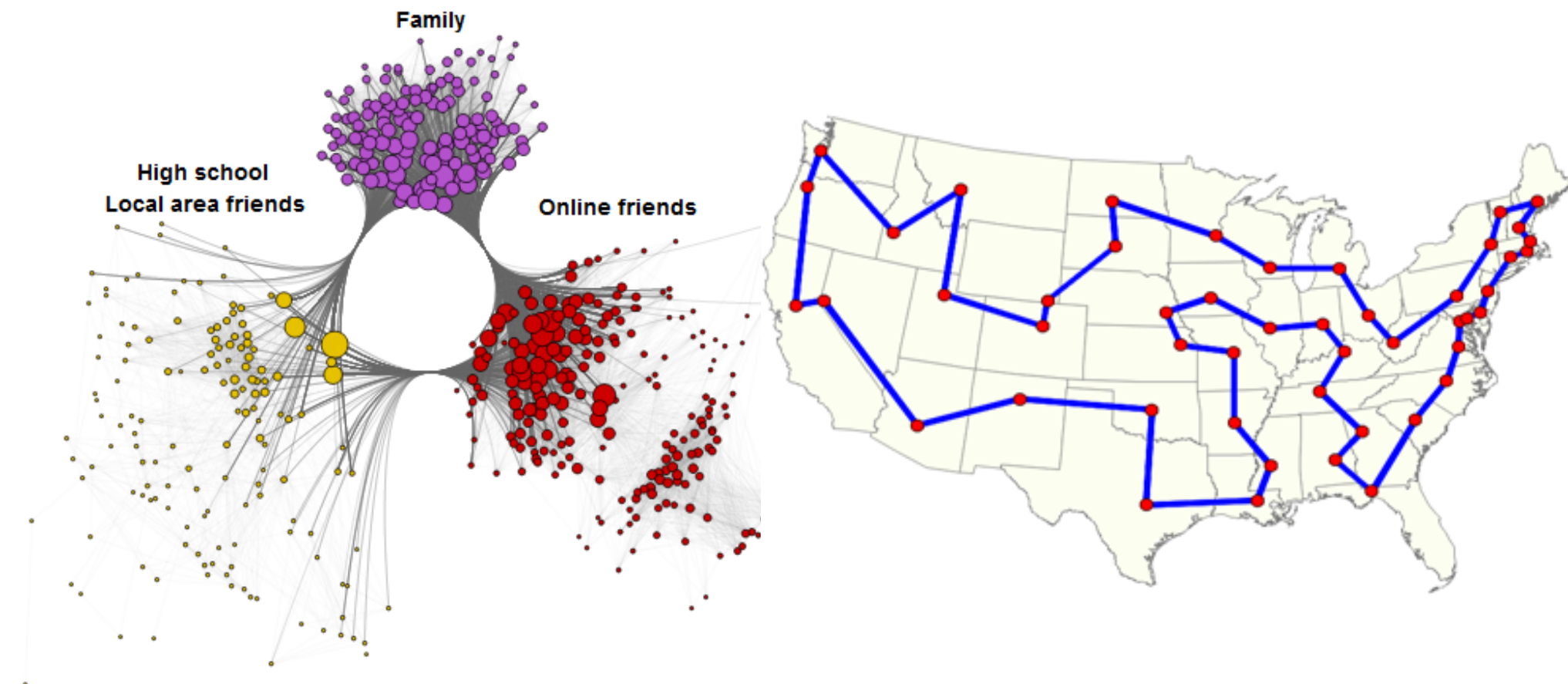
Graph neural network is a rising concept in machine learning that has demonstrated significant potential in many areas. Our project aims to further explore this potential by applying it to the classic **Traveling Salesman Problem (TSP)**. We explore a novel concept called **imitation learning**, which is a special type of **reinforcement learning**.

The former works focuses on fixed size training where the problem size is about 500 hundreds of constraints with 1000 of variables. In this work, we focus on generalization ability instead. Specifically, we train on TSP with 10 and 15 nodes, which translate into an **Integer Linear Programming (ILP)** with hundreds of variables and several hundreds of constraints. Then, we test on TSP instances with more nodes and see the generalization ability of this novel approach.

The Basics of GNN

A **graph neural network (GNN)** is a class of neural network for processing data best represented by graph structures. They were popularized by their use in supervised learning on properties of various molecules from the natural of this task: 3d graph structure.

Since their inception, variants of the message passing neural network (MPNN) framework have been proposed. These models optimize GNNs for use on larger graphs and apply them to domains such as social networks, citation networks, and online communities. GNNs have also been relatively successful in various NP-hard combinatorial problems, automated planning and path-planning areas due to the inherent graph structure of data.



Problem Formulation

The **classical TSP** is given in a very simple form: *Given n nodes with their coordinates, find the closed tour which goes through all the nodes with the shortest length.*

One way to formulate TSP is to model which by **Integer Linear Programming (ILP)**. Given an undirected weighted graph $\mathcal{G} = (\mathcal{E}, \mathcal{V})$, we label the nodes with numbers $1, \dots, n$ and define

$$x_{ij} := \begin{cases} 1, & \text{if } (i, j) \in \mathcal{E}' \\ 0, & \text{if } (i, j) \in \mathcal{E} \setminus \mathcal{E}', \end{cases}$$

where $\mathcal{E}' \subset \mathcal{E}$ is a variable which can be viewed as a compact representation of all variables x_{ij} . This means that x_{ij} acts like an **indicator** variable, it's 1 if it's in the optimal tour, 0 otherwise. We denote the weight on edge (i, j) by c_{ij} , then for a TSP problem instance, we can formulate the problem by so-called **Miller–Tucker–Zemlin formulation**[3].

One way to solve an ILP is to first **relax** the integrality constraints by letting $x_{ij} \in [0, 1]$. Then, we can divide the original relaxed LP into two sub-problems by **splitting the feasible region** according to a **chosen variable**

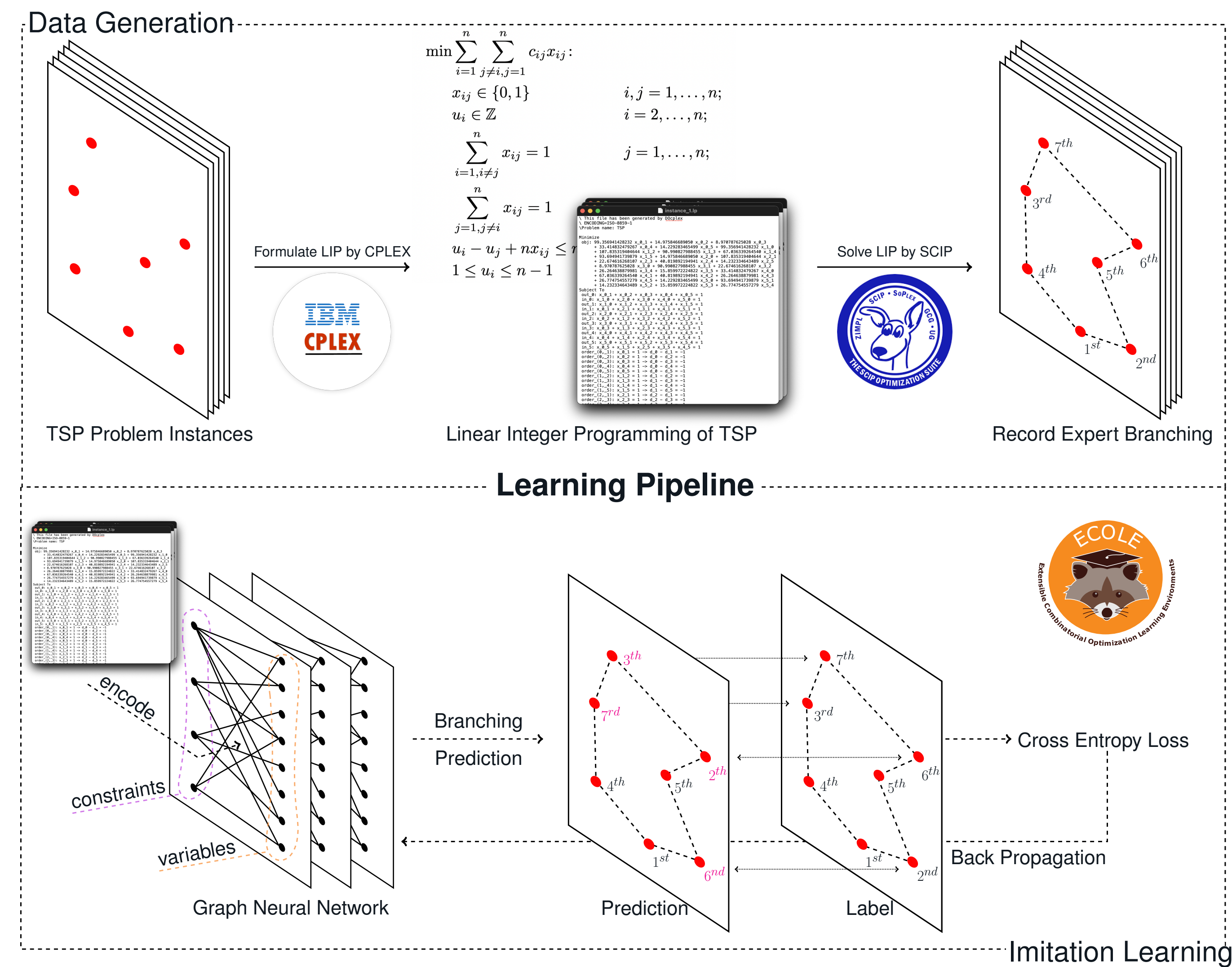
that violates integrality constraints in the current relaxed LP solution x^* , say x_i^* , then

$$x_i \leq \lfloor x_i^* \rfloor \vee x_i \geq \lceil x_i^* \rceil, \quad \exists i \leq p \mid x_i^* \notin \mathbb{Z}.$$

are two additional constraints in two sub-problems respectively. This gives us a recursive algorithm called **Branch and Bound**[6]. Note that a good branching decision can reduce the solving time significantly, hence, our plan is simple, we want to solve TSP by branch and bound and learn how to branch. This is a sequential decision problem, hence, we naturally model this as a **Markov Decision Process (MDP)** problem[2].

Learning Pipeline

Our **learning pipeline** is as follows. We first create some random TSP instances, and turn it into ILP. Then, we use imitation learning[1] to learn how to choose the **branching target** at each branching.



Specifically, we will pass TSP instances to SCIP, which is a modern SOTA MILP (Mixed Integer Linear Programming) solver, to get all the solving states in order to solve MDP problem. The modern solver usually use mixed branching strategy to balance the running time, and a well-known costly but strong strategy is called **strong branching**, and in order to learn the strongest branching strategy, we make SCIP to use strong branch with probability a half. This learning process is called **imitation learning**, or **behavioral cloning**[4].

Mathematically, we first run the SOTA solver SCIP to get state-action pairs $\mathcal{D} = \{(s_i, \mathbf{a}_i^*)\}_{i=1}^N$, where the states is the solver's branching strategy, and the action set contains what variables we can branch on.

To learn the policy $\tilde{\pi}^*$, we minimize the cross-entropy loss between our branching prediction and the solver's branching choice:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{(s, \mathbf{a}^*) \in \mathcal{D}} \log \tilde{\pi}_{\theta}(\mathbf{a}^* \mid s).$$

After learning, we evaluate our model on TSP instances with various sizes to see the generalization ability. Specifically, we use Ecole[5] to do the evaluation. We configure SCIP to its default strategy and compare the result to our learned branching strategy by looking at the time needed to solve this particular TSP instance.

Experimental Results

Wait for the evaluation result

Conclusion

Unlike other works which focuses on just to solve TSP approximately, we focus on generalization ability of solving TSP **exactly**.

With limited computation power and training on small size instances, our result shows a promising performance on generalization ability of branch, which is a hard combinatorial optimization problem and typically hard to generalize.

The **significance** of our works is pretty straightforward: We're now confidence to generalize on larger instances by combining various machine learning techniques like standard reinforcement learning, deep learning, transformer, etc. This is promising since our result shows that we can fit small instances while keep the generalization ability.

discuss

Future Work

Due to computational limitation, we can't train on larger instances. One may further generalize our work by training on larger instances, for instance, TSP50.

On the other hand, our work should be easy to reproduce and be combined with various of standard reinforcement technique to further fine-tuned our trained model.